

LAB - 0

1)

1. The address of the main function is **0x10150**. It is because the first instruction of my code was executed at **0x00000000000010150**
2.
 - a. **auipc t0, 0x1 & addi t0, t0, 800**: These instructions calculated the memory address for **num1** and stored it in register **t0**.
 - b. **lw t1, 0(t0)**: This loaded the value 15 (**hex 0xf**) from memory into register **t1**.
 - c. **auipc & addi** :These calculated the memory address for **num2** and updated **t0**.
 - d. **lw t2, 0(t0)**: This loaded the value 10 (**hex 0xa**) from memory into register **t2**.
 - e. **mul t3, t1, t2**: This multiplied the values in **t1 (15)** and **t2 (10)** and stored the result 150 (**hex 0x96**) into register **t3**.
 - f. **sw t3, 0(t0)**: This stored the value 150 from **t3** back into the memory address held in **t0**.
3. Address of num1: **0x11470**

Calculation: The instruction was **auipc t0, 0x1** at PC **0x10150**. **0x10150 + 0x1000 = 0x11150**. Then **addi** added 800 (which is **0x320** in hex). **0x11150 + 0x320 = 0x11470**.

Address of num2: **0x11474** (4 bytes after num1)

Address of result: **0x11478** (4 bytes after num2)

4. Yes, there are quite differences.
 - **Difference**: The **la** (Load Address) instruction that I wrote in program. S does not appear in the dump file. Instead, it has been replaced by two instructions: **auipc** and **addi**.

```

root@04c4c0c97f3c:/home/os-iitm/work# spike -d pk my_RISCV_executable
(spike) until pc 0 0000000000010150
tbl loader
(spike)
core 0: 0x0000000000010150 (0x00001297) auipc    t0, 0x1
(spike)
core 0: 0x0000000000010154 (0x32028293) addi     t0, t0, 800
(spike)
core 0: 0x0000000000010158 (0x0002a303) lw      t1, 0(t0)
(spike)
core 0: 0x000000000001015c (0x00001297) auipc    t0, 0x1
(spike)
core 0: 0x0000000000010160 (0x31828293) addi     t0, t0, 792
(spike)
core 0: 0x0000000000010164 (0x0002a383) lw      t2, 0(t0)
(spike)
core 0: 0x0000000000010168 (0x02730e33) mul     t3, t1, t2
(spike)
core 0: 0x000000000001016c (0x80818293) addi     t0, gp, -2040
(spike)
core 0: 0x0000000000010170 (0x01c2a023) sw      t3, 0(t0)
(spike)
core 0: 0x0000000000010174 (0x05d00893) li      a7, 93
(spike)
core 0: 0x0000000000010178 (0x00000073) ecall
core 0: exception trap_user_ecall, epc 0x0000000000010178
(spike) reg 0 t3
0x0000000000000096
(spike) reg 0 t0
0x00000000000011478
(spike) until pc 0 0000000000010150
root@04c4c0c97f3c:/home/os-iitm/work# spike -d pk my_RISCV_executable
(spike) until pc 0 0000000000010150
tbl loader
(spike)
core 0: 0x0000000000010150 (0x00001297) auipc    t0, 0x1
(spike)
core 0: 0x0000000000010154 (0x32028293) addi     t0, t0, 800
(spike) reg 0 t0
0x00000000000011470
(spike) █

```

|| .

1 .

Intermediate results are stored in **temporary registers** such as **a4** and **a5**.
 These registers are used to hold values loaded from memory before they are moved into argument registers (**a0**, **a1**) for function calls.

2.

The final result is stored in register **a0** at the end of **main()**.

According to the RISC-V calling convention, the return value of a function is placed in **a0**, and **main()** returns this value.

3.

The original assembly code is short and simple because it directly uses registers and does not use the stack or function calls.

On the other hand, the assembly code generated from the **C** program has more instructions. It creates a stack frame, stores local variables in memory, and handles function calls.

Therefore, the compiler-generated assembly code is longer and more detailed, while the handwritten assembly code is simpler and more efficient.

```

$sh: Command not found: riscv64-unknown-elf-gcc
sahitibheri@Sahitis-MacBook-Air:~$ docker run --platform linux/amd64 -v ~/riscv:/home/os-iitm/work svkv/riscv-tools:v1.0
root@eeee311cbd95:/home/os-iitm# ls
install work
root@eeee311cbd95:/home/os-iitm# cd work
root@eeee311cbd95:/home/os-iitm/work# ls
a.out hello.s mul.c my_RISCV_executable my_dump_file my_dump_file2 program.S
root@eeee311cbd95:/home/os-iitm/work# riscv64-unknown-elf-gcc -O0 -g -o my_RISCV_executable mul.c
root@eeee311cbd95:/home/os-iitm/work# riscv64-unknown-elf-objdump -d my_RISCV_executable | grep "<mul>"
000000000010150 <mul>:
    1015a: fb7ff0ef          jal    ra,10150 <mul>
root@eeee311cbd95:/home/os-iitm/work# riscv64-unknown-elf-objdump -d my_RISCV_executable | grep "<main>"
100fa: 080000ef          jal    ra,1017a <main>
00000000001017a <main>:
root@eeee311cbd95:/home/os-iitm/work# spike -d pk my_RISCV_executable
(spike)
core 0: 0x0000000000001000 (0x0000297) auipc    t0, 0x0
(spike) until pc 0 00000000001017a
bbl loader
(spike)
core 0: 0x000000000001017a (0x0001101) c.addi    sp, -32
(spike)
core 0: 0x000000000001017c (0x000ec06) c.sdsp    ra, 24(sp)
(spike)
core 0: 0x000000000001017e (0x000e822) c.sdsp    s0, 16(sp)
(spike)
core 0: 0x0000000000010180 (0x0001000) c.addi4spn s0, sp, 32
(spike)
core 0: 0x0000000000010182 (0x00047d1) c.li      a5, 20
(spike)
core 0: 0x0000000000010184 (0xfef42623) sw      a5, -20(s0)
(spike)
core 0: 0x0000000000010188 (0x00047a9) c.li      a5, 10
(spike)
core 0: 0x000000000001018a (0xfef42423) sw      a5, -24(s0)
(spike)
core 0: 0x000000000001018e (0xfe842703) lw      a4, -24(s0)
(spike)
core 0: 0x0000000000010192 (0xfec42783) lw      a5, -20(s0)
(spike)
core 0: 0x0000000000010196 (0x00085ba) c.mv      a1, a4
(spike)
core 0: 0x0000000000010198 (0x000853e) c.mv      a0, a5
(spike) q
root@eeee311cbd95:/home/os-iitm/work# riscv64-unknown-elf-objdump -D a.out > my_dump_file
root@eeee311cbd95:/home/os-iitm/work# spike -d pk my_RISCV_executable
(spike) until pc 0 000000000010150
bbl loader
(spike)
core 0: 0x0000000000010150 (0x0001101) c.addi    sp, -32
(spike)
core 0: 0x0000000000010152 (0x000ec22) c.sdsp    s0, 24(sp)
(spike)
core 0: 0x0000000000010154 (0x0001000) c.addi4spn s0, sp, 32
(spike)
core 0: 0x0000000000010156 (0x00087aa) c.mv      a5, a0
(spike)
core 0: 0x0000000000010158 (0x000872e) c.mv      a4, a1
(spike)

```